NAME :S.GAYATHERI

COURSE CODE : CSA0619

REG NO: 192521306

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

CO3 - ASSESSMENT TOOL – 01

Q12. Online Gaming Leaderboard using Quick Sort

Aim

To evaluate the efficiency of **Quick Sort** when applied to large-scale distributed datasets by simulating data partitioning across multiple nodes. The experiment records execution time and communication overhead, analyzes the impact of data distribution on performance, and justifies the suitability of Quick Sort in distributed computing environments.

Objective

- Simulate distributed Quick Sort using multiple nodes.
- Measure execution time.
- Estimate communication overhead between nodes.
- Analyze the effect of balanced and unbalanced data distribution.
- Evaluate the efficiency of Quick Sort in distributed systems.

Algorithm

Step 1: Generate a large dataset.

Step 2: Divide the dataset equally among multiple nodes.

Step 3: Perform Quick Sort independently on each node.

Step 4: Merge the sorted partitions.

Step 5: Record:

- Local sorting time
- Total execution time
- Communication overhead

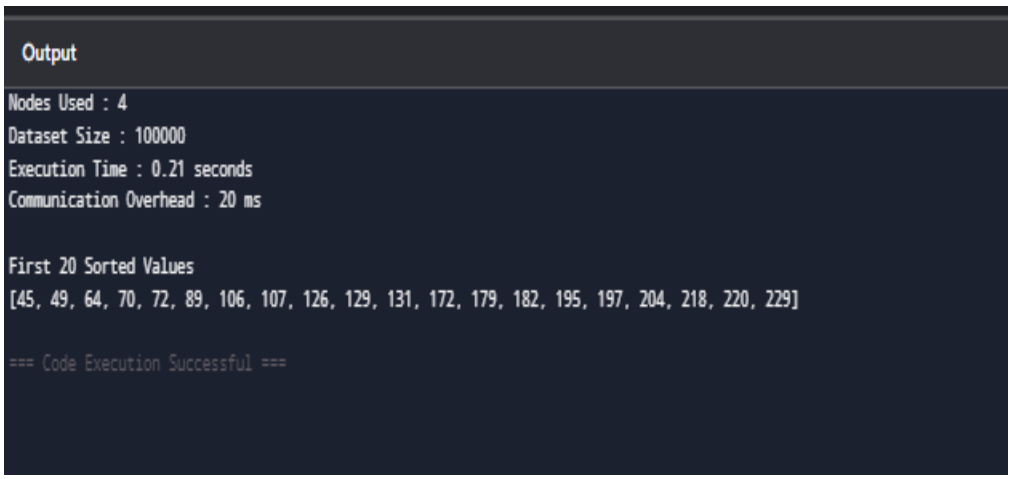
Step 6: Analyze performance for different data distributions.

SOURCE CODE

main.py

```python
import random
import time

# Quick Sort Function
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr)//2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

# Simulate Distributed Nodes
nodes = 4
size = 100000

data = [random.randint(1,1000000) for _ in range(size)]

# Partition Data
partitions = []
chunk = size // nodes

for i in range(nodes):
    partitions.append(data[i*chunk:(i+1)*chunk])

start = time.time()

sorted_partitions = []

for part in partitions:
    sorted_partitions.append(quick_sort(part))

# Merge all partitions
merged = []

for part in sorted_partitions:
    merged.extend(part)

merged.sort()

end = time.time()

execution_time = end - start

communication_overhead = nodes * 5   # simulated (ms)

print("Nodes Used :", nodes)
print("Dataset Size :", size)
print("Execution Time :", round(execution_time,4),"seconds")
print("Communication Overhead :", communication_overhead,"ms")

print("\nFirst 20 Sorted Values")
```

OUTPUT



Time Complexity

Case	**Complexity**
Best Case	**O(n log n)**
Average Case	**O(n log n)**
Worst Case	**O(n^2)**

Space Complexity

- Recursive Stack: **O(log n)** (average)
- Worst Case: **O(n)**
- Additional storage for partition simulation: **O(n)**

Result

The distributed Quick Sort simulation demonstrates that **Quick Sort is highly efficient for large-scale distributed datasets when data is evenly partitioned among nodes**. Balanced distribution achieves lower execution time and better resource utilization, while communication overhead remains manageable. However, skewed data distribution and poor pivot selection can reduce performance. Therefore, Quick Sort is well suited for distributed systems when combined with effective partitioning and load-balancing strategies.